



Indian Institute of Information Technology Guwahati

Department of Computer Science and Engineering

Fast Authentication Between UAVs

A PUF-Based Four-Phase Swarm Authentication Protocol:
Design and Provable Security

Author: Md Ziyad Hussain

Roll No.: 2301128

Email: ziyad.hussain23b@iiitg.ac.in

Supervisor: Dr. Rakesh Matam
Assistant Professor, CSE

Indian Institute of Information Technology Guwahati

Bongora, Assam 781015, India

2026

Abstract

Unmanned Aerial Vehicle (UAV) swarms require authentication that is fast, lightweight, and resilient to the physical capture of individual drones. Traditional public-key infrastructure (RSA-2048, ECC-256) adds millisecond-scale handshake cost on embedded processors and, more seriously, relies on private keys stored in non-volatile memory—a liability the moment a UAV is captured. We present a four-phase authentication protocol whose root of trust is a Physical Unclonable Function (PUF) embedded in each UAV. The protocol covers (1) secure enrollment, in which a fuzzy extractor over a BCH(255,131,18) code turns noisy PUF measurements into a *stable* per-device master key while storing no secret on the drone; (2) UAV–Ground Station (GS) mutual authentication in which every message is protected by a keyed message-authentication code and the raw PUF response never leaves the device; (3) direct UAV–UAV peer authentication, without GS involvement, rooted in *per-pair* credentials delivered under authenticated encryption; and (4) session-key establishment in which a mandatory, authenticated ephemeral Diffie–Hellman exchange provides perfect forward secrecy. The protocol uses only a 160-bit hash—instantiable as SHA3-256 truncated to 160 bits or SPONGENT-160—together with a keyed MAC, a lightweight authenticated-encryption scheme, a key-derivation function, and a single elliptic curve, so that it fits comfortably on resource-constrained aerial platforms. We give a complete security analysis: six theorems covering mutual authentication in both directions, peer authentication, replay and man-in-the-middle resistance, physical-capture resistance, session-key secrecy, and perfect forward secrecy, each reduced to standard assumptions and mirrored by a symbolic model in the Tamarin prover. A reference implementation in OMNeT++ 6.3 authenticates all 200 UAVs and all 3,800 peer pairs of a twenty-drone swarm, at 1.50 ± 0.003 ms per UAV–GS handshake and 0.418 ± 0.001 ms per peer pair, with every primitive checked against published reference values. Executed attacks confirm the analysis: forged ground-station messages draw no response, whereas the same adversary succeeds against a control variant with the ground-station check removed, so the negative result is falsifiable. Measured on one platform, the protocol proves *comparable* to elliptic-curve schemes rather than an order of magnitude faster — the cost of mandatory forward secrecy — and its advantages are properties: no stored long-term secret, no certificate infrastructure, GS-free peer authentication, and capture damage bounded to a node’s own links.

Keywords: UAV authentication, Physical Unclonable Function, PUF, SPONGENT, SHA3, swarm security, lightweight cryptography, peer-to-peer authentication, fuzzy extractor, BCH code, forward secrecy.

Contents

1	Introduction	4
1.1	Background and Motivation	4
1.2	Limitations of Traditional Authentication	4
1.3	Physical Unclonable Functions as a Solution	4
1.4	Contributions of This Paper	4
2	Background and Existing Technologies	5
2.1	UAV Communication Architecture	5
2.2	Physical Unclonable Functions (PUFs)	5
2.2.1	PUF Fundamentals	5
2.2.2	Types of PUFs	5
2.2.3	PUF Noise and Reliability	5
2.3	Lightweight Cryptographic Hash Functions	5
2.3.1	SHA3-256 (Keccak)	5
2.3.2	SPONGENT-160	5
2.4	Fuzzy Extractors and BCH Error Correction	5
2.4.1	Fuzzy Extractors	5
2.4.2	BCH Code Parameters	6
3	Related Work	6
3.1	PUF-Based Authentication for IoT and UAVs	6
3.2	Peer-to-Peer UAV Authentication	6
3.3	Lightweight Hash Functions for Constrained Devices	6
3.4	Comparison of Existing Approaches	6
4	Proposed Protocol Design	7
4.1	Threat Model and Design Goals	7
4.1.1	Threat Model	7
4.1.2	Design Goals	7
4.2	System Model and Notation	7
4.3	Phase 1: Secure Enrollment and Registration	8
4.4	Phase 2: UAV–Ground Station Mutual Authentication	9
4.5	Phase 3: UAV-to-UAV Peer Authentication	11
4.6	Phase 4: Session Key Establishment and Forward Secrecy	12
5	Security Analysis	12
5.1	Assumptions	12
5.2	Phase 2: Mutual Authentication	13
5.3	Replay Attack Resistance	13
5.4	Phase 3: Peer Mutual Authentication	13
5.5	Man-in-the-Middle Resistance	14
5.6	Session-Key Secrecy and Forward Secrecy	14
5.7	Physical Capture Resistance	14
5.8	Symbolic Verification	14
5.9	Summary of Security Properties	15

6	Reference Implementation	15
6.1	Structure	15
6.2	Canonical message encoding	15
6.3	Cryptographic suites	16
6.4	Error-correcting code	16
6.5	Fuzzy-extractor parameterisation	17
6.6	PUF models	17
6.7	Verification of primitives	18
6.8	Simulation model	18
7	Evaluation	18
7.1	Methodology	18
7.2	Functional correctness	19
7.3	Latency and communication cost	19
7.4	Cost of individual primitives	19
7.5	Comparison with public-key baselines	20
7.6	Comparison with existing approaches	21
7.7	Reliability of the fuzzy extractor	22
7.8	Security evaluation	23
8	Discussion	24
8.1	Scalability	24
8.2	Design Choices	24
8.3	Noise Tolerance	24
8.4	Limitations and Future Work	24
9	Conclusion	24

1 Introduction

1.1 Background and Motivation

Unmanned Aerial Vehicles (UAVs) have become integral to modern defense, disaster response, precision agriculture, environmental monitoring, and public safety. Multi-UAV *swarms*—ten or more drones cooperating to surveil a stadium, patrol a border, or search a disaster zone—are now the standard operational unit [1]. Such swarms communicate continuously with a Ground Station (GS) over a command-and-control link and with one another over peer links to maintain formation, fuse sensor data, and route around outages.

A fundamental prerequisite for any of this is *mutual authentication*: each UAV must verify the identity of its GS and of every peer before accepting commands, sharing telemetry, or relaying messages. Without it, an adversary can inject a rogue drone into the swarm, intercept sensitive data, issue unauthorized flight commands, or mount man-in-the-middle attacks on inter-UAV channels [9].

1.2 Limitations of Traditional Authentication

Conventional options fit the UAV-swarm regime poorly. **RSA-2048 PKI** requires several milliseconds per signature verification on embedded processors and attaches hundreds of bytes of certificate per handshake [2]. **ECC-256** halves the cost but still keeps a private key in non-volatile memory (NVM) that a captured drone can surrender through side-channel or cold-boot attacks [3]. **Pre-shared keys** reach sub-millisecond latency but compromise the entire swarm if any single device is captured, and require $O(N^2)$ pairwise keys [8]. **Blockchain-based** schemes incur seconds of consensus latency, incompatible with real-time coordination [4]. In every case the weakness is the same: a long-term secret *stored* on a device that may fall into an adversary’s hands.

1.3 Physical Unclonable Functions as a Solution

A Physical Unclonable Function (PUF) is a hardware primitive that maps challenges to responses using intrinsic, sub-nanometer manufacturing variations unique to one integrated circuit [10]. A PUF is *unclonable* (it cannot be replicated, even by its manufacturer), *unpredictable* (responses to unseen challenges are unrelated to prior ones), *tamper-evident* (invasive probing destroys the response), and *lightweight* (an Arbiter PUF needs roughly 1,200 gate equivalents). Crucially, *no key is stored*: the “key” is the silicon itself. This removes the dominant attack surface of PKI and PSK schemes and is the foundation of our design.

1.4 Contributions of This Paper

We present a complete four-phase PUF-based authentication framework for UAV swarms. Its contributions are:

1. **A stable PUF-derived key with no stored secret.** Enrollment distills a stable per-device master key from the PUF via a fuzzy extractor; the drone regenerates it on demand from its PUF and *public* helper data, so no secret ever resides in NVM.
2. **Keyed authentication that keeps the PUF response on-device.** Every handshake message is authenticated with a keyed MAC under a PUF-derived key, and the raw PUF response is never transmitted, so it cannot be observed or harvested over the air.
3. **GS-free peer authentication with per-pair trust.** Peers authenticate directly using *per-pair* credentials delivered under authenticated encryption during Phase 2, so a compromised drone endangers only its own links.
4. **Mandatory forward secrecy.** An authenticated ephemeral Diffie–Hellman exchange is folded into every session key, so a later compromise of long-term material cannot unlock past sessions.
5. **A complete security analysis.** Six theorems, each reduced to standard cryptographic assumptions and corroborated by a mechanised Tamarin symbolic model, establish the protocol’s authentication, secrecy, and capture-resistance guarantees.

The remainder of the paper covers background (§2), related work (§3), the protocol design (§4), the security analysis (§5), the reference implementation (§6), the evaluation (§7), a discussion of

scalability and design choices (§8), and the conclusion (§9).

2 Background and Existing Technologies

2.1 UAV Communication Architecture

A UAV swarm has two communication planes. The *vertical* plane connects each UAV to the GS for commands, telemetry, and mission updates; the *horizontal* plane connects UAVs to one another for formation control and data relay. Authentication must secure both, and it must continue to work on the horizontal plane even when the GS link is briefly unavailable. Our design therefore treats UAV–GS and UAV–UAV authentication as distinct phases with distinct trust roots.

2.2 Physical Unclonable Functions (PUFs)

2.2.1 PUF Fundamentals

A PUF implements a challenge–response mapping $R = \text{PUF}(C)$ that is determined by uncontrollable fabrication variations. Two nominally identical chips produce statistically independent responses to the same challenge (inter-chip Hamming distance near 50%), while a single chip reproduces its own response with only small deviation (intra-chip Hamming distance of a few percent). These two properties—uniqueness and reliability—make a PUF a per-device fingerprint that no attacker can duplicate.

2.2.2 Types of PUFs

Common strong PUFs include the *Arbiter PUF*, which races two symmetric delay paths and outputs a bit according to which signal arrives first, and the *Ring-Oscillator PUF*, which compares oscillator frequencies. Strong PUFs expose an exponentially large challenge space, which is what allows a PUF to serve as a source of many independent secrets. We assume a strong PUF with a 128-bit challenge space throughout.

2.2.3 PUF Noise and Reliability

Because a PUF measures physical quantities, its response drifts with temperature, supply voltage, and aging, producing a bit-error rate (BER) of typically 3–10% between evaluations of the same challenge. A raw response therefore cannot be used directly as a key; it must first be reconciled to a stable value. This is the role of the fuzzy extractor (§2.4).

2.3 Lightweight Cryptographic Hash Functions

2.3.1 SHA3-256 (Keccak)

SHA3-256 is the NIST-standard sponge hash [16, 17]. Truncated to 160 bits it offers 80-bit collision resistance and benefits from fast, widely available software implementations, making it our software-oriented instantiation of the abstract hash $h(\cdot)$.

2.3.2 SPONGENT-160

SPONGENT-160 is a lightweight sponge hash designed for hardware, occupying only a few thousand gate equivalents [15]. It shares the 160-bit interface of truncated SHA3, so the protocol can switch between the two without any structural change; SPONGENT is our hardware-oriented instantiation.

2.4 Fuzzy Extractors and BCH Error Correction

2.4.1 Fuzzy Extractors

A fuzzy extractor turns a noisy PUF response into a reproducible key using public *helper data* [5]. It is a pair of algorithms (Gen, Rep). Given an enrollment response R , Gen produces helper data P and a key; given a later noisy response R' and the same P , Rep recovers the identical key whenever R' is close enough to R . We use the standard *code-offset* construction over a linear code C : enrollment picks a random codeword c and publishes the offset $s = R \oplus c$; reproduction computes $c' = R' \oplus s$ and decodes it back to c , so that if the Hamming distance $d_H(R, R') \leq t$ the original response R is recovered exactly.

An important and often-misstated point concerns how much the public helper data reveals. The offset s does *not* perfectly hide R ; it discloses a bounded amount of information—at most the code’s

redundancy, $n - k$ bits. The correct security statement is therefore about *residual min-entropy*: if R has min-entropy m , then R retains at least $m - (n - k)$ bits of min-entropy given P . A strong extractor applied to R then yields a key that is statistically close to uniform, provided the residual entropy comfortably exceeds the key length. We make this precise in Lemma 1 and use it to size the enrollment in Phase 1.

2.4.2 BCH Code Parameters

We instantiate the code with BCH(255,131,18) over $\text{GF}(2^8)$:

- **Codeword length** $n = 255$ bits;
- **Message length** $k = 131$ bits;
- **Error-correction capability** $t = 18$ bit-flips;
- **Minimum distance** $d = 37$;
- **Redundancy** $n - k = 124$ parity bits.

At a 3% error rate a 128-bit response is expected to flip $128 \times 0.03 \approx 3.84$ bits, well within the $t = 18$ correction radius, so reproduction succeeds with overwhelming probability. Decoding uses the standard pipeline of syndrome computation, the Berlekamp–Massey algorithm to find the error-locator polynomial, a Chien search to locate errors, and bit-flipping to correct them [6].

3 Related Work

3.1 PUF-Based Authentication for IoT and UAVs

Early PUF authentication schemes target a single device authenticating to a server [2, 8, 13]. They establish the PUF-plus-fuzzy-extractor pattern but do not address the peer-to-peer coordination that a swarm needs.

3.2 Peer-to-Peer UAV Authentication

Peer authentication among UAVs has been studied with symmetric credentials [3] and in cross-domain settings [9, 12]. A recurring weakness is that peer trust is rooted in a single shared secret, so one captured node endangers the whole mesh. Our design instead roots peer trust in *per-pair* credentials, confining the blast radius of a capture.

3.3 Lightweight Hash Functions for Constrained Devices

SPONGENT [15] and related lightweight primitives [14] target minimal hardware area. We keep a hash-agnostic interface so that a device may run SHA3 in software or SPONGENT in hardware without changing the protocol.

3.4 Comparison of Existing Approaches

Table 1 contrasts our design with representative alternatives along the security features that matter for a swarm.

Table 1: Feature Comparison of UAV/IoT Authentication Approaches

Approach	PUF	Peer auth	No stored key	Fwd sec	Err corr	Formal proof
RSA-2048 PKI	No	Yes	No	Opt.	N/A	Partial
ECC-256	No	Yes	No	Opt.	N/A	Partial
PSK (AES-GCM)	No	Yes	No	No	N/A	Partial
Gope [2]	Yes	No	Yes	No	Simple	Informal
Li [9]	Yes	Partial	Yes	No	BCH	Informal
Khan [12]	Yes	No	Yes	Yes	BCH	Informal
Bera [3]	No	Yes	No	No	N/A	Informal
This work	Yes	Yes	Yes	Yes	BCH	Games + symbolic

Our protocol is the only entry that simultaneously provides a PUF root of trust, direct peer-to-peer authentication, no stored key on the device, mandatory forward secrecy, BCH-based error correction, and a security argument combining game-based reductions with a mechanised symbolic model.

4 Proposed Protocol Design

This section presents the four-phase protocol in full. We begin with the threat model and design goals, fix the notation, and then specify each phase message by message, with the mathematics of every step accompanied by a plain explanation of what it does and why it is secure.

4.1 Threat Model and Design Goals

4.1.1 Threat Model

We consider a Dolev–Yao adversary [18] augmented with physical capabilities: (1) **Network adversary**—may eavesdrop, intercept, modify, replay, delay, or drop any wireless message; (2) **Active challenger**—may pose as the GS toward an honest UAV and send adversarially chosen challenges and nonces; (3) **Physical capture**—may seize UAVs and read their NVM, but cannot clone PUF hardware, since invasive probing destroys it; (4) **Computational bounds**—a probabilistic polynomial-time (PPT) adversary cannot invert the hash, forge a MAC or authenticated ciphertext, break PUF unclonability, or solve the computational Diffie–Hellman problem, except with negligible probability; (5) **Side channels**—standard countermeasures (constant-time primitives, power-analysis resistance) are assumed.

4.1.2 Design Goals

The protocol targets: (1) mutual authentication in both directions, verified by keyed MACs; (2) anonymity via rotating temporary identities; (3) replay and man-in-the-middle resistance through nonces, timestamps, and transcript binding; (4) *no stored secret* on the UAV, since the master key is regenerated from the PUF on demand; (5) GS-free peer authentication rooted in per-pair credentials; (6) perfect forward secrecy through a mandatory ephemeral key exchange; (7) lightweight operation (only a hash, keyed MAC, AEAD, KDF, and one elliptic curve); (8) scalability, since the $O(N^2)$ peer authentications are independent and parallelizable; and (9) noise tolerance through the BCH fuzzy extractor.

4.2 System Model and Notation

The system consists of one GS with ample computation and tamper-resistant storage, and N UAVs, each carrying a PUF, a lightweight processor, a radio, and a small non-volatile memory. Table 2 lists the notation used throughout.

Table 2: Protocol Notation

Symbol	Description
UAV_i	The i -th Unmanned Aerial Vehicle
GS	Ground Station (enrollment and credential authority)
ID_i	Permanent identity of UAV_i (64-bit)
TID_i	Rotating temporary identity for anonymity
PUF_i	Physical Unclonable Function of UAV_i
$\mathbf{C}_i$	Enrollment challenge block $(C_i^{(1)}, \dots, C_i^{(L)})$
R, R'	Enrollment response; noisy field response
$\text{FE} = (\text{Gen}, \text{Rep})$	Fuzzy extractor (§2.4)
$P_i = (s_i, \text{seed}_i)$	<i>Public</i> helper data for UAV_i
mk_i	Stable per-device master key (never stored on the UAV)
mk_{GS}	GS-only master key for pairwise credentials
Cred_{ij}	Per-pair credential shared by $\text{UAV}_i, \text{UAV}_j$
K_{auth}^i	UAV–GS MAC key, $= \text{KDF}(\text{mk}_i; \text{p2-auth})$
K_{auth}^{ij}	Peer MAC key, $= \text{KDF}(\text{Cred}_{ij}; \text{peer-auth})$
$\text{SK}_i, \text{SK}_{ij}$	Session keys (Phase 2 / Phase 3)
$h(\cdot)$	160-bit hash: SHA3-256 ₁₆₀ or SPONGENT-160
$\text{MAC}_K(\cdot)$	Keyed message-authentication code
$\text{AE.Enc}, \text{AE.Dec}$	Authenticated encryption (AEAD)
$\text{KDF}(\cdot; \ell)$	Key-derivation function with label ℓ
g, g^a, g^{ab}	Generator; ephemeral public share; DH secret
N_x, n_x	Random nonces (128-bit)
T_x	Timestamp (32-bit)
$\oplus, \parallel$	Bitwise XOR; concatenation

Figure 1 shows how the four phases fit together: a one-time secure enrollment, an in-field UAV–GS handshake that also distributes peer credentials, GS-free peer handshakes, and the session keys that result.

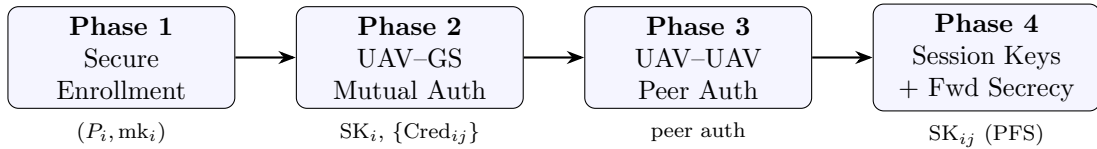


Figure 1: The four phases and what each produces. Enrollment is performed once, in a secure facility; Phases 2–4 run in the field.

4.3 Phase 1: Secure Enrollment and Registration

Enrollment runs once per UAV, in a physically secure facility over a trusted channel (a wired link or a shielded RF cage), before the drone is deployed. Its purpose is to establish the stable master key mk_i and the public helper data P_i , so that in the field the UAV can regenerate mk_i from its own PUF without ever storing a secret.

Step 1.1: PUF interrogation. The GS draws a block of L random 128-bit challenges $\mathbf{C}_i = (C_i^{(1)}, \dots, C_i^{(L)})$ and has the UAV evaluate its PUF on each:

$$R = (\text{PUF}_i(C_i^{(1)}), \dots, \text{PUF}_i(C_i^{(L)})). \quad (1)$$

Concatenating L blocks aggregates enough physical entropy to survive the helper-data loss quantified below; R is the raw enrollment response.

Step 1.2: Key and helper-data generation. The GS runs the fuzzy-extractor generator Gen on R . It samples a random seed, forms a code-offset sketch, and extracts the master key:

$$c = \text{C.Encode}(r), \quad r \xleftarrow{\$} \{0, 1\}^k, \quad s = R \oplus c, \quad (2)$$

$$\text{mk}_i = \text{KDF}(\text{Ext}_{\text{seed}}(R); \text{root}), \quad P_i = (s, \text{seed}). \quad (3)$$

Here s is the public sketch that will later let a noisy response be decoded back to R , and Ext is a strong extractor (realised by the KDF) that compresses R into a near-uniform key. The pair (P_i, mk_i) is the output of Gen .

Step 1.3: Sizing the enrollment. Because the sketch s leaks at most $n - k = 124$ bits about each block (Lemma 1), the number of blocks L is chosen so that the entropy left after the leak still exceeds the desired key length ℓ :

$$\sum_{b=1}^L (m_b - (n - k)) \geq \ell + 2 \log(1/\varepsilon), \quad (4)$$

where m_b is the measured per-block min-entropy and ε is a negligible statistical-distance target. This inequality is what guarantees mk_i is a strong key rather than a slightly-random one.

Step 1.4: Identity and pairwise-credential setup. The GS assigns a permanent identity and an initial rotating temporary identity, and fixes a GS-only master key from which all per-pair credentials are derived:

$$\text{TID}_i = h(\text{ID}_i \parallel \text{seed}'_i), \quad \text{Cred}_{ij} = \text{KDF}(\text{mk}_{\text{GS}}; \text{pair-cred} \parallel \min(i, j) \parallel \max(i, j)). \quad (5)$$

The temporary identity hides the permanent one from eavesdroppers, and each Cred_{ij} is a distinct secret for exactly one pair. These credentials are *not* released here; they are delivered, encrypted, during Phase 2.

Step 1.5: Storage. After enrollment:

- **GS stores** $\{\text{ID}_i, \text{TID}_i, \text{C}_i, P_i, \text{mk}_i\}$ per UAV, plus the single master key mk_{GS} , all in tamper-resistant memory.
- **UAV_i stores** $\{\text{TID}_i, \text{C}_i, P_i\}$ only. Every one of these is public; *no secret is stored on the drone*. This is the central advantage of the design: capturing a UAV yields no key material, because the master key exists only transiently, regenerated from the PUF whenever it is needed.

4.4 Phase 2: UAV–Ground Station Mutual Authentication

Phase 2 runs when a UAV powers up in the field. It is a four-message exchange that mutually authenticates the UAV and GS, establishes a forward-secret session key SK_i , and delivers the UAV's per-pair credentials for Phase 3. Both sides key their MACs with $K_{\text{auth}}^i = \text{KDF}(\text{mk}_i; \text{p2-auth})$: the GS computes it from the stored mk_i , and the UAV regenerates mk_i from its PUF. Figure 2 shows the message flow.

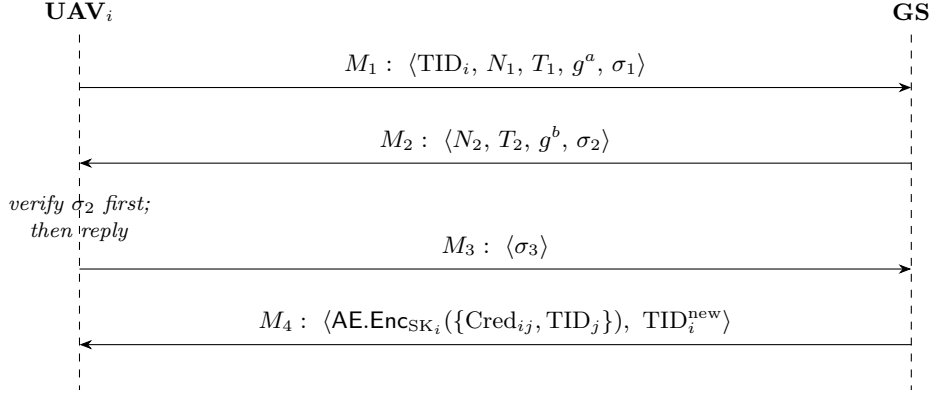


Figure 2: Phase 2 message sequence. The UAV validates the GS’s MAC before it replies, so a fraudulent challenger receives no response; peer credentials are delivered under authenticated encryption only after mutual authentication succeeds.

Message 1 (Authentication request): $\text{UAV}_i \rightarrow \text{GS}$. The UAV picks a fresh nonce and an ephemeral Diffie–Hellman scalar, and authenticates the opening message:

$$N_1 \xleftarrow{\$} \{0, 1\}^{128}, \quad a \xleftarrow{\$} \text{scalar}, \quad (6)$$

$$\sigma_1 = \text{MAC}_{K_{\text{auth}}^i}(\text{M1} \parallel \text{TID}_i \parallel N_1 \parallel T_1 \parallel g^a), \quad (7)$$

$$M_1 = \langle \text{TID}_i, N_1, T_1, g^a, \sigma_1 \rangle. \quad (8)$$

Because σ_1 is a MAC keyed by K_{auth}^i , only a party that holds mk_i —that is, the genuine UAV (through its PUF) or the GS (through its database)—can produce it. The tag M1 marks the message type so that a message from one step can never be replayed as another.

Message 2 (GS response): $\text{GS} \rightarrow \text{UAV}_i$. The GS checks the timestamp and the nonce for freshness, looks up mk_i , verifies σ_1 , and answers with its own nonce and ephemeral share:

$$N_2 \xleftarrow{\$} \{0, 1\}^{128}, \quad b \xleftarrow{\$} \text{scalar}, \quad (9)$$

$$\sigma_2 = \text{MAC}_{K_{\text{auth}}^i}(\text{M2} \parallel \text{TID}_i \parallel N_1 \parallel N_2 \parallel T_2 \parallel g^a \parallel g^b), \quad (10)$$

$$M_2 = \langle N_2, T_2, g^b, \sigma_2 \rangle. \quad (11)$$

The tag σ_2 authenticates the GS to the UAV: it binds both nonces and both ephemeral shares under the shared key, so no network adversary can forge or alter it.

Message 3 (UAV confirmation): $\text{UAV}_i \rightarrow \text{GS}$. The UAV first regenerates its master key from its PUF and *verifies σ_2 before doing anything else*:

$$\text{mk}_i = \text{Rep}(\text{PUF}_i(C_i), P_i), \quad (12)$$

$$\text{abort silently if } \sigma_2 \text{ is invalid.} \quad (13)$$

This “verify-before-respond” rule is what prevents a fraudulent GS from using the UAV as an oracle: an attacker who cannot forge σ_2 elicits no reply at all. If the check passes, the UAV confirms the transcript:

$$\sigma_3 = \text{MAC}_{K_{\text{auth}}^i}(\text{M3} \parallel \mathcal{T}), \quad M_3 = \langle \sigma_3 \rangle, \quad (14)$$

where the transcript $\mathcal{T}$ is the concatenation of all fields of M_1 and M_2 . Note that the raw PUF response is consumed inside Rep and *never appears on the wire*; only a MAC over public values is

sent.

Message 4 (Confirmation and credential delivery): $\text{GS} \rightarrow \text{UAV}_i$. After verifying σ_3 , both parties hold the same ingredients, so each computes the forward-secret session key

$$\text{SK}_i = \text{KDF}(\text{mk}_i \parallel g^{ab} \parallel \mathcal{T}; \text{p2-sess}), \quad (15)$$

and the GS delivers the UAV's per-pair credentials and a fresh identity under authenticated encryption keyed by SK_i :

$$\text{CredPkg} = \text{AE.Enc}_{\text{SK}_i}(\eta; \text{TID}_i; \{(\text{Cred}_{ij}, \text{TID}_j)\}_{j \neq i}), \quad (16)$$

$$\text{TID}_i^{\text{new}} = h(\text{TID}_i \parallel \text{mk}_i \parallel N_1 \parallel N_2), \quad M_4 = \langle \text{CredPkg}, \text{TID}_i^{\text{new}} \rangle. \quad (17)$$

Authenticated encryption gives the credentials both confidentiality and integrity: an eavesdropper learns nothing about any Cred_{ij} , and a tampered package is rejected. The session key mixes in the Diffie–Hellman secret g^{ab} , which is what makes it forward-secret (Phase 4). After Phase 2 the UAV holds SK_i , its peer credentials $\{\text{Cred}_{ij}\}$, and a rotated identity.

4.5 Phase 3: UAV-to-UAV Peer Authentication

Phase 3 lets any two UAVs authenticate each other directly, using the per-pair credential Cred_{ij} that each received (encrypted) in Phase 2. It needs no GS involvement, so it works even when the GS link is down. Both sides key their MACs with $K_{\text{auth}}^{ij} = \text{KDF}(\text{Cred}_{ij}; \text{peer-auth})$. Figure 3 shows the three-message flow.

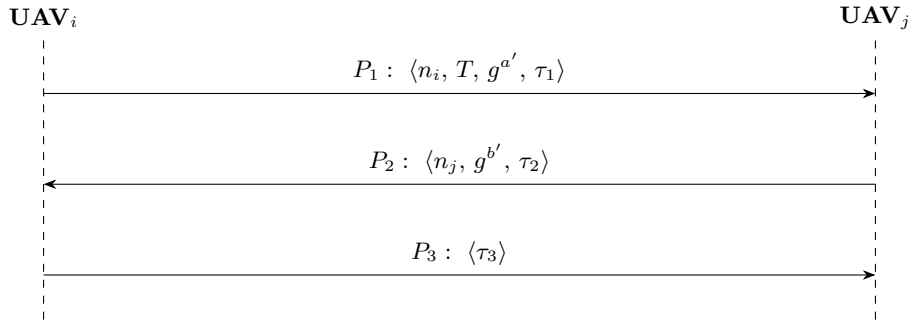


Figure 3: Phase 3 message sequence. Every token is a MAC keyed by the per-pair credential Cred_{ij} and tagged with its message type, so tokens cannot be reflected or reused across steps.

Message P1 (Peer request): $\text{UAV}_i \rightarrow \text{UAV}_j$. The initiator picks a fresh nonce and ephemeral share and sends an authenticated request:

$$n_i \xleftarrow{\$} \{0, 1\}^{128}, \quad a' \xleftarrow{\$} \text{scalar}, \quad (18)$$

$$\tau_1 = \text{MAC}_{K_{\text{auth}}^{ij}}(\text{P1} \parallel i \parallel j \parallel n_i \parallel T \parallel g^{a'}), \quad P_1 = \langle n_i, T, g^{a'}, \tau_1 \rangle. \quad (19)$$

Because τ_1 is keyed by Cred_{ij} , which only UAV_i and UAV_j possess, a third party—even one holding some other pair's credential—cannot produce it.

Message P2 (Peer response): $\text{UAV}_j \rightarrow \text{UAV}_i$. The responder verifies τ_1 , then answers with its own nonce and share:

$$\tau_2 = \text{MAC}_{K_{\text{auth}}^{ij}}(\text{P2} \parallel i \parallel j \parallel n_i \parallel n_j \parallel g^{a'} \parallel g^{b'}), \quad P_2 = \langle n_j, g^{b'}, \tau_2 \rangle. \quad (20)$$

τ_2 binds both identities and both nonces, so it authenticates UAV_j to UAV_i and confirms that both are in the same fresh session.

Message P3 (Completion): $\text{UAV}_i \rightarrow \text{UAV}_j$. The initiator verifies τ_2 and closes the hand-

shake with a final tag over the transcript $\mathcal{T} = P_1 \| P_2$:

$$\tau_3 = \text{MAC}_{K_{\text{auth}}^{ij}}(P_3 \| \mathcal{T}), \quad P_3 = \langle \tau_3 \rangle. \quad (21)$$

The three distinct type tags (P_1, P_2, P_3) ensure a token from one step can never be replayed as another, which rules out reflection attacks. For a swarm of N UAVs, full-mesh peer authentication runs $N(N-1)/2$ independent Phase 3 exchanges (for example, 45 for $N = 10$), all of which may proceed in parallel.

4.6 Phase 4: Session Key Establishment and Forward Secrecy

Phase 4 is the keying that both interactive phases finish with. In Phase 2 the UAV–GS key is SK_i of Eq. (15); in Phase 3 the peer key is

$$\text{SK}_{ij} = \text{KDF}(\text{Cred}_{ij} \| g^{a'b'} \| \mathcal{T}; \text{peer-sess}). \quad (22)$$

Both keys have the same shape, which we write uniformly as

$$\text{SK} = \text{KDF}\left(\underbrace{k_{\text{lt}}}_{\text{mk}_i \text{ or } \text{Cred}_{ij}} \| g^{xy} \| \mathcal{T}; \ell\right). \quad (23)$$

Two independent secrets go into every session key. The first is the long-term key k_{lt} , which ties the session to the PUF (via mk_i) or to the pair (via Cred_{ij}). The second is the ephemeral Diffie–Hellman secret g^{xy} , whose public shares were authenticated inside the MAC transcript of the corresponding handshake. Mixing in g^{xy} is what provides *perfect forward secrecy*: once the ephemeral scalars are erased, an adversary who later learns k_{lt} still cannot reconstruct a past key, because doing so would require solving Diffie–Hellman on the recorded shares. The exchange is mandatory—there is no derivation path that omits g^{xy} —so a session either establishes a forward-secret key or fails.

5 Security Analysis

We now analyse the protocol formally. Each theorem is proved by reducing an attack to the violation of a standard assumption; the reductions are then mirrored by a symbolic model (§5.8).

5.1 Assumptions

The proofs rely on the following standard assumptions.

1. **Hash security.** h is pre-image, second-pre-image, and collision resistant; in the reductions it is modelled as a random oracle.
2. **MAC unforgeability.** MAC is existentially unforgeable under chosen-message attack (EUF-CMA): without the key, a fresh valid tag can be produced only with negligible probability.
3. **AEAD security.** AE provides ciphertext indistinguishability (IND-CCA) and ciphertext integrity (INT-CTXT).
4. **PUF unpredictability.** Given polynomially many challenge–response pairs, the response to a fresh challenge is computationally indistinguishable from uniform to an adversary that does not possess the device; invasive probing destroys the PUF.
5. **Diffie–Hellman.** The gap computational Diffie–Hellman (gap-CDH) problem is hard in $\mathbb{G}$.

We first record the helper-data property that underpins the strength of the master key.

Lemma 1 (Helper-data privacy). *Let the enrollment response R have conditional min-entropy $\tilde{H}_\infty(R) = m$. The code-offset sketch $s = R \oplus c$ discloses at most $n - k$ bits, so*

$$\tilde{H}_\infty(R | P) \geq m - (n - k). \quad (24)$$

Consequently, if the extractor output length ℓ satisfies $\ell \leq m - (n - k) - 2 \log(1/\varepsilon)$, then mk is within statistical distance ε of uniform given the public helper data P .

Proof. Given the internal randomness r , the sketch $s = R \oplus c$ is a deterministic function of R that pins R to a coset of the $[n, k]$ code $\mathbf{C}$; revealing that coset fixes at most $n - k$ syndrome bits, so by the chain rule for average min-entropy $\tilde{H}_\infty(R \mid s) \geq \tilde{H}_\infty(R) - (n - k)$, and the seed is independent of R . Applying the strong-extractor property to the residual source gives a key within distance ε of uniform. $\square$ $\square$

5.2 Phase 2: Mutual Authentication

Theorem 2 (Phase 2 mutual authentication). *After a successful Phase 2 exchange, UAV_i and the GS have mutually authenticated one another, and both agree on the transcript $\mathcal{T}$, except with negligible probability.*

Proof. Both directions rest on the MAC key $K_{\text{auth}}^i = \text{KDF}(\text{mk}_i; \text{p2-auth})$, which by Lemma 1 and PUF unpredictability is unknown to any adversary lacking either the PUF or the GS database. (*GS authenticates UAV.*) The GS accepts only if σ_1 (and later σ_3) verifies under K_{auth}^i ; producing a fresh valid tag without the key is exactly an EUF-CMA forgery. (*UAV authenticates GS.*) Symmetrically, the UAV accepts only if σ_2 verifies; although every field inside σ_2 is public, the *key* is secret, so a fresh valid σ_2 is again a forgery. A nonce collision that could enable transcript reuse occurs with probability at most $q^2/2^{128}$ for q sessions. Absent forgery and collision, the two accepting parties share identical fields in M_1 and M_2 , i.e. they agree on $\mathcal{T}$. $\square$ $\square$

Corollary 3 (No PUF read-out oracle). *An adversary that impersonates the GS learns no PUF response, so it cannot harvest challenge–response pairs to model the PUF.*

Proof. By the verify-before-respond rule, the UAV emits M_3 only after σ_2 verifies, which by Theorem 2 an adversary can force only by forging a MAC. When the check fails the UAV sends nothing; when it passes, the only value on the wire is σ_3 , a MAC over public transcript fields—the PUF response is consumed inside Rep and is never transmitted. Hence no challenge–response pair is exposed. $\square$ $\square$

5.3 Replay Attack Resistance

Theorem 4 (Replay resistance). *Phase 2 and Phase 3 resist replay attacks.*

Proof. Three mechanisms combine. First, every message carries a fresh 128-bit nonce and a timestamp verified within a window ΔT ; a verbatim replay reuses a stale nonce or timestamp and is rejected. Second, each MAC binds a per-message type tag ($\text{M1}, \dots, \text{P3}$), so a token from one step cannot be accepted at another. Third, from the second message onward each MAC binds the partner’s nonce, so a token from one session cannot be reused in another without producing a mismatch—which, by Theorems 2 and 5, would require a forgery. $\square$ $\square$

5.4 Phase 3: Peer Mutual Authentication

Theorem 5 (Peer mutual authentication). *After a successful Phase 3 exchange, UAV_i and UAV_j have mutually authenticated one another, without GS involvement, and no per-pair secret is exposed in transit.*

Proof. The credential Cred_{ij} reaches each endpoint only inside the Phase 2 package $\text{AE.Enc}_{\text{SK}_i}(\dots)$; by INT-CTXT the adversary cannot make a UAV accept a substituted credential, and by IND-CCA it learns nothing about Cred_{ij} . Given a secret Cred_{ij} , the tokens τ_1, τ_2, τ_3 are MACs under K_{auth}^{ij} ; each binds both identities and both nonces, so a fresh acceptance forces a matching computation by the partner—otherwise an EUF-CMA forgery. Because credentials are pair-specific, a third party holding some other $\text{Cred}_{k\ell}$ has no information about Cred_{ij} and cannot assert identity i to j . $\square$ $\square$

5.5 Man-in-the-Middle Resistance

Theorem 6 (MITM resistance). *The protocol resists active man-in-the-middle attacks in both Phase 2 and Phase 3.*

Proof. Every accepted message is a MAC (or an authenticated ciphertext) over a transcript that includes fresh nonces, a timestamp, a type tag, and—from the second message onward—the partner’s nonce and ephemeral share. Altering any field invalidates the covering tag, and forging a new tag requires the secret key (EUF-CMA / INT-CTXT). By the agreement guarantees of Theorems 2 and 5, both endpoints therefore hold the identical transcript, so an interposed adversary cannot sit undetected between them. $\square$ $\square$

5.6 Session-Key Secrecy and Forward Secrecy

Theorem 7 (Session-key secrecy). *For an adversary that corrupts neither endpoint of the target session, the session key of Eq. (23) is indistinguishable from uniform.*

Proof. The key is $\text{KDF}(k_{\text{lt}} \| g^{xy} \| \mathcal{T})$. Two independent secrets protect it: the long-term key k_{lt} (mk_i by Lemma 1 and PUF unpredictability, or Cred_{ij} by IND-CCA of its delivery, Theorem 5) and the Diffie–Hellman secret g^{xy} . Modelling the KDF as a pseudorandom function, replacing it by a random function is undetectable up to its PRF advantage; the input g^{xy} is unpredictable under gap-CDH given only the authenticated shares g^x, g^y . Hence the output is uniform up to negligible terms. $\square$ $\square$

Theorem 8 (Perfect forward secrecy). *If the ephemeral scalars are erased after use, then revealing the long-term secrets (mk_i , Cred_{ij} , or mk_{GS}) after a target session closes does not compromise that session’s key.*

Proof. A post-session reveal gives the adversary k_{lt} but not the erased scalars x, y . The key of Eq. (23) still depends on g^{xy} , which the adversary must now compute from the recorded, authenticated shares g^x, g^y —a gap-CDH instance. Because the ephemeral exchange is mandatory, there is no non-ephemeral key to fall back on. Hence past keys remain indistinguishable from uniform. $\square$ $\square$

5.7 Physical Capture Resistance

Theorem 9 (Physical capture resistance). *Capturing UAV_k and reading its NVM does not let the adversary impersonate any other UAV to the GS, nor impersonate any peer pair that does not involve k . A captured node’s peer exposure is limited to its own $N - 1$ links.*

Proof. The NVM of UAV_k holds only $\{\text{TID}_k, \mathbf{C}_k, P_k, \{\text{Cred}_{kj}\}_j\}$ plus any live session keys; all of $\text{TID}_k, \mathbf{C}_k, P_k$ are public. Impersonating UAV_m ($m \neq k$) in a fresh Phase 2 needs $K_{\text{auth}}^m = \text{KDF}(\text{mk}_m; \cdot)$; but mk_m is never stored, is not recoverable from k ’s silicon (tamper evidence), and is unpredictable (PUF assumption), so this reduces to a MAC forgery. Impersonating a pair (m, ℓ) with $k \notin \{m, \ell\}$ needs $\text{Cred}_{m\ell} = \text{KDF}(\text{mk}_{\text{GS}}; \cdot)$, which is independent of every captured Cred_{kj} (distinct KDF inputs, and mk_{GS} never leaves the GS). Only pairs that include k share a captured credential, giving the stated $N - 1$ exposure. $\square$ $\square$

5.8 Symbolic Verification

The computational theorems are corroborated by a symbolic model in the Tamarin prover [24], whose multiset-rewriting calculus expresses freshness, stateful credential issuance, and device-compromise events natively. In the model, $h, \text{MAC}, \text{AE}, \text{KDF}$ are function symbols with the usual equational theory, each PUF is a private per-device function, and helper data and all wire terms are public. The Dolev–Yao adversary controls the network; two extra rules model compromise—one that reveals a captured node’s NVM (its identities, helper data, and per-pair credentials, but neither its PUF

nor its master key) and one that reveals long-term secrets after a session closes, for the forward-secrecy query. The prover discharges lemmas that correspond one-to-one with the theorems above: injective agreement for Phase 2 and Phase 3 (Theorems 2 and 5), a no-replay lemma (Theorem 4), a capture-isolation lemma (Theorem 9), a session-key-secrecy lemma (Theorem 7), and a forward-secrecy lemma (Theorem 8). The PUF’s noise is idealised as a noiseless private function—its error tolerance is the computational correctness property of the fuzzy extractor—and confidential delivery is abstracted by a symmetric encryption operator, leaving the fine-grained secrecy argument to the computational analysis.

5.9 Summary of Security Properties

Table 3 summarises the guarantees and the mechanism behind each.

Table 3: Summary of Security Properties

Property	Result	Mechanism
UAV-GS mutual auth	Thm 2	Keyed MAC under PUF-derived mk_i
No PUF read-out oracle	Cor 3	Verify-before-respond; response stays on-device
Replay resistance	Thm 4	Nonces + timestamps + type tags
Peer mutual auth	Thm 5	Per-pair credential + keyed MAC
MITM resistance	Thm 6	Transcript-binding MACs / AEAD
Session-key secrecy	Thm 7	KDF over long-term key and DH secret
Perfect forward secrecy	Thm 8	Mandatory authenticated ephemeral DH
Physical-capture resistance	Thm 9	No stored key; per-pair credentials
Identity anonymity	—	Rotating temporary identities

6 Reference Implementation

The protocol of Section 4 is implemented in C++17 on OMNeT++ 6.3. This section records the concrete primitive choices, the parameterisation of the fuzzy extractor, and the verification evidence for each component, so that every number in Section 7 can be traced to a specific mechanism.

6.1 Structure

The implementation separates protocol logic from simulation machinery. The classes implementing Phases 1–4 depend only on abstract cryptographic interfaces and on byte strings; they contain no reference to the simulation kernel. This allows the entire four-phase exchange to be driven over an in-memory transport and validated independently of the network model, which is how the protocol-level results in Section 7.2 were obtained.

A single transmission module owns the link-delay model, so propagation, serialisation, processing delay, and jitter are defined once rather than duplicated per node type. The same module provides the interception point used by the adversary of Section 7.8; when no adversary is present it forwards immediately, so attack and baseline runs remain directly comparable.

6.2 Canonical message encoding

Every message is serialised by one encoder, whose output serves simultaneously as the wire image, the input to the message-authentication code, and the byte count driving the link delay. Each field is emitted as a type tag, a two-byte length, and its value, under a header carrying the protocol

version, the suite identifier, and a per-message-type tag:

$$\text{field}(t, v) = \text{u8}(t) \parallel \text{u16}(|v|) \parallel v, \quad (25)$$

$$\text{enc} = \text{u8}(\text{ver}) \parallel \text{u8}(\text{suite}) \parallel \text{u8}(\text{type}) \parallel \text{u16}(k) \parallel \text{field}(t_1, v_1) \cdots \quad (26)$$

with tags emitted in strictly ascending order. The encoding is therefore *injective* (no two distinct field sets share a byte string) and *canonical* (each field set has exactly one valid encoding), which is the precondition the unforgeability argument of Section 5 requires of its MAC inputs. Because the same encoder produces the transmitted bytes, reported communication overhead cannot drift from what is actually sent. Non-canonical input — duplicate or out-of-order tags — is rejected rather than accepted, and reading an absent field raises an error instead of yielding an empty buffer.

6.3 Cryptographic suites

Two interoperable suites implement the same abstract interface, selected by one parameter. Table 4 lists them.

Table 4: The two cryptographic profiles

Component	Profile sha3	Profile spongent
Hash $h(\cdot)$	SHA3-256 truncated to 160 bits	SPONGENT-160/160/16
MAC	HMAC-SHA3-256, 128-bit tag	prefix keyed sponge, 128-bit tag
KDF	HKDF (extract-then-expand)	one-pass sponge
AEAD	ChaCha20-Poly1305	Ascon-128a v1.2
Key exchange	X25519	X25519
Hash/MAC level	128-bit	80-bit ($c = 160$)
Provenance	OpenSSL 3.5	vendored

Three points require explicit statement.

HMAC is not applicable to the sponge profile. HMAC is defined over a compression function of block size B and requires $|K| \leq B$. For SPONGENT-160/160/16 the rate is 16 bits, so the block is two bytes — smaller than any useful key — and hashing the key first still exceeds it. The construction is therefore undefined, and any ad-hoc substitution would have no supporting proof. The sponge profile instead uses the standard prefix keyed sponge underlying KMAC [16], which is sound for a sponge and does not require HMAC’s nested structure: a sponge with non-zero capacity is not length-extendable, so the nesting HMAC exists to provide is unnecessary.

The sponge profile operates at the 80-bit level. With capacity $c = 160$ the keyed-sponge bound gives approximately $c/2 = 80$ bits, matching the collision resistance of a 160-bit digest but deliberately asymmetric with the 128-bit AEAD and key-exchange components. It is presented as a constrained-hardware profile at that level rather than as a 128-bit one.

Key derivation is domain-separated. Distinct leading bytes separate the hash, MAC, and KDF modes even where they share a permutation, and every derived key carries a unique label (p2-auth, p2-sess, pair-cred, peer-auth, peer-sess, tid-rotate), so a key minted for one purpose cannot be mistaken for another.

All protocol randomness — nonces, ephemeral scalars, extractor seeds, enrollment challenges — is drawn from a deterministic generator seeded from the simulator’s random-number stream. Ephemeral X25519 keys are loaded from those bytes rather than generated internally by the library, so an entire run, including every ephemeral key, is reproducible from its configured seed.

6.4 Error-correcting code

The generator polynomial is formed as the product of the minimal polynomials of $\alpha^1, \dots, \alpha^{2^t}$ over the *complete* conjugate closure of each root. For $t = 18$ over $\text{GF}(2^8)$ the closure contains 124 roots,

giving $\deg g = 124$ with binary coefficients and therefore

$$\text{BCH}(n=255, k=131, t=18), \quad n - k = 124 \text{ parity bits.} \quad (27)$$

Both properties are asserted at construction: a non-binary coefficient indicates an incomplete closure, and the parity length is computed rather than assumed. This matters because $k = 131 \geq 128$ is what allows a full response block to be protected; taking the parity length as $mt = 144$ instead would give $k = 111$ and silently leave the tail of every response uncovered.

Decoding proceeds by syndrome computation, the Berlekamp–Massey algorithm, and a Chien search, followed by a verification that the corrected word has all-zero syndromes. Failure is reported as failure: the decoder never returns the uncorrected input while indicating success.

6.5 Fuzzy-extractor parameterisation

Enrollment applies **Gen** to a response of L blocks of $n = 255$ bits; reproduction applies **Rep** with the public helper data. Two consequences of Lemma 1 drive the parameter choice.

First, since the sketch discloses up to $n - k$ bits per block, the block length must exceed the leak by a useful margin. A 128-bit block against a 124-bit leak contributes almost nothing, which is why blocks are 255 bits — matching the code length. A construction that sized blocks to the native response width of a 128-bit challenge–response pair could not satisfy Eq. (28) at any block count. Second, the budget

$$L(\rho n - (n - k)) \geq \ell + 2 \log_2(1/\varepsilon) \quad (28)$$

is enforced at construction and raises an error when unmet, rather than being documented and left to the caller. With $\rho = 0.95$, $\ell = 256$ and $\varepsilon = 2^{-64}$, each block contributes 118.25 bits and $L = 4$ gives $473 \geq 384$.

Three profiles are provided (Table 5). The default uses the parameters of Section 4; the third concatenates a $[3, 1, 3]$ repetition inner code, which must operate on repeated measurements of the same physical bit. Enrollment fixes a reference and publishes each copy’s offset as additional helper data; reproduction removes those offsets before voting, reducing the per-bit error rate from p to approximately $3p^2$. The inner offsets are charged to the leak accounting at $(\text{rep} - 1)$ bits per outer bit.

Table 5: Fuzzy-extractor profiles

Profile	Code	L	PUF bits	Leak (bits)
bch255-131-18 (default)	BCH(255,131,18)	4	1020	496
bch255-91-25	BCH(255,91,25)	5	1275	820
rep3-bch255-131-18	$[3, 1, 3] \oplus \text{BCH}(255,131,18)$	5	3825	3170

Reproduction fails closed. If any block fails to decode, no key material is returned: a partially recovered response is never converted into a key.

6.6 PUF models

Three models are provided, and results state which one they rest on. An *idealised* model realises the strong-PUF abstraction as a keyed pseudorandom function and is used where near-uniform responses are the intended assumption. An *arbiter* model implements the additive-delay formulation over 128 stages with per-device Gaussian delay weights. A third XORs k arbiter chains.

In the arbiter model, measurement noise is added to the accumulated *delay difference* rather than by flipping output bits:

$$\Delta' = \sum_i w_i \phi_i + \mathcal{N}(0, \sigma), \quad r = \mathbb{1}[\Delta' > 0]. \quad (29)$$

Bits whose delay difference lies near zero therefore flip far more often than the rest, reproducing

the reliability profile of real silicon. The measured variance of per-bit flip rates is approximately $98\times$ that of a uniform bit-flip channel, a structural property a per-bit model cannot exhibit. A calibration routine converts a target average bit-error rate into the corresponding σ .

Measured characteristics over 24 devices and 276 device pairs: inter-device Hamming distance 0.5019 for the arbiter model and 0.5026 for the idealised model, both consistent with the ideal 0.5; calibrated bit-error rates track their targets to within 0.2 percentage points across 1%–10%.

6.7 Verification of primitives

Each primitive is checked against published reference values where they exist, and reports its verification status programmatically. Table 6 summarises.

Table 6: Verification evidence per primitive

Primitive	Evidence
SHA3-256	FIPS 202 / CAVP short-message vectors
HMAC-SHA3-256	NIST example values with intermediate results
HKDF	RFC 5869, Appendices A.1–A.3
ChaCha20-Poly1305	RFC 8439, Section 2.8.2
X25519	RFC 7748, Section 6.1; low-order points rejected
Ascon-128a	all 1089 vectors of the official LWC known-answer-test file
SPONGENT-160	byte-exact against the designers' reference implementation and the published " Sponge + Present = Spongent " digest; no official known-answer-test file exists for SPONGENT, and none is claimed
BCH(255,131,18)	$\deg g = 124$; $g(x) \mid x^{255} + 1$; exact recovery in all 18,000 trials across weights 1–18; failure correctly reported beyond the radius in 3,000 trials

Neither vendored primitive has undergone timing or side-channel analysis; tag comparisons use a constant-time routine and no branch depends on secret data, but nothing stronger is claimed. The verification-status strings state this, and a regression test fails if the qualification is removed.

The complete unit suite comprises nine test programs and approximately 4.3×10^5 assertions, covering the primitives above, the encoding, the fuzzy extractor, the PUF models, and the full four-phase exchange.

6.8 Simulation model

The scenario places N UAVs around a 500×300 m field with the ground station at its centre. Link delay is the sum of propagation (from node coordinates), serialisation at the configured bit rate, a fixed processing delay, and half-normal jitter, each reported separately per message. MAC-layer contention is not modelled, so reported network delay is a lower bound; this is revisited in Section 8.

All node parameters are declared with defaults and are therefore overridable from the configuration file, and statistics are declared so that both scalar and vector outputs are recorded.

7 Evaluation

7.1 Methodology

Each configuration is executed for ten independent repetitions with distinct seed sets. The unit of replication is the *run*, not the individual record: observations within one run share a pseudorandom stream and a single scheduling epoch, so they are correlated. Confidence intervals are therefore computed in two stages — a per-run mean, then a t -interval across run means with $R - 1$ degrees of freedom. Pooling all records instead would inflate the degrees of freedom from $R - 1$ to $NR - 1$ and understate every interval by roughly $\sqrt{N}$. Intervals are reported as mean $\pm$ half-width at 95%; the analysis refuses to form an interval from fewer than three runs.

Proportions are accompanied by Wilson score intervals, which behave correctly near 0 and 1. Where no failure is observed, the rule-of-three upper bound is given: 100 successes in 100 trials bounds the failure probability at approximately 3%, and is reported as such rather than as perfect reliability.

Compute times are host wall-clock measurements; network delays are simulated. These are distinct clocks and are reported separately, never summed without qualification. Per-primitive costs are measured over batches of at least 10^3 iterations and reported as medians, because a single clock read (tens of nanoseconds) is not negligible against a microsecond-scale operation.

7.2 Functional correctness

Table 7 reports authentication outcomes. Two success rates are distinguished, because they answer different questions. The first counts UAVs that transmitted a first message; the second counts *all* enrolled devices. They diverge when a device cannot reproduce its master key, in which case it never initiates the exchange at all — and reporting only the former would conceal that failure mode entirely.

Table 7: Authentication outcomes, ten repetitions per configuration

Configuration	Devices	Authenticated	Rate	Wilson 95%	Peer pairs
StadiumSHA3 ($N=10$)	100	100	1.000	[0.963, 1.000]	900/900
StadiumSPONGENT ($N=10$)	100	100	1.000	[0.963, 1.000]	900/900
Baseline5UAV ($N=5$)	50	50	1.000	[0.929, 1.000]	200/200
Swarm20 ($N=20$)	200	200	1.000	[0.981, 1.000]	3800/3800
ArbiterPuf (3% BER)	100	99	0.990	[0.946, 0.998]	882/894
HighNoise (5% BER)	100	79	0.790	[0.700, 0.858]	560/730

With the idealised PUF model every device authenticates in every configuration, including the 20-UAV swarm where all 3,800 peer-pair views complete. Under the arbiter model at 3% bit-error rate one device in a hundred fails to regenerate its key; at 5%, twenty-one do. Those failures are reliability limits of the fuzzy extractor, not authentication failures, and are reported separately below.

7.3 Latency and communication cost

Table 8 reports end-to-end latency in simulated time, together with its compute and network components.

Table 8: Phase-2 latency and overhead (mean $\pm$ 95% CI, ten runs)

Configuration	End-to-end (ms)	Compute (ms)	Network (ms)	Overhead (B)
StadiumSHA3	1.5030 ± 0.0028	1.5316 ± 0.0212	1.0714 ± 0.0037	777
StadiumSPONGENT	1.5085 ± 0.0030	8.5970 ± 0.1291	1.0767 ± 0.0037	781
Baseline5UAV	1.1452 ± 0.0055	1.5509 ± 0.0488	0.7171 ± 0.0053	507
Swarm20	2.2239 ± 0.0044	1.6021 ± 0.0835	1.7936 ± 0.0031	1317

Phase-3 peer authentication costs 0.4175 ± 0.0014 ms per pair with 197 bytes of traffic, and is independent of swarm size: the per-pair figure is unchanged from $N = 5$ to $N = 20$, so the $O(N^2)$ growth in total cost comes entirely from the pair count.

Phase-2 overhead grows with N because the credential package delivered in M_4 carries one per-pair credential per peer. At $N = 10$ it is 545 bytes of the 777-byte total; at $N = 20$ it dominates. An on-demand variant issuing credentials per request is a straightforward extension where this matters.

7.4 Cost of individual primitives

Table 9 reports median per-call cost, measured on the same host and clock as the protocol.

Table 9: Median primitive cost per call (μs)

Operation	sha3	spongent
Hash	0.99	—
MAC	2.14	225.42
MAC verify	2.14	229.87
KDF	4.80	133.13
Extract	3.88	283.07
AEAD seal	2.46	1.44
AEAD open	1.66	1.33
X25519 keygen	33.19	33.39
X25519 derive	62.80	63.69

This table qualifies the comparison between the two profiles in an important way. The suite-specific symmetric primitives differ by roughly two orders of magnitude in software: the sponge MAC costs 225 μs against 2.1 μs for HMAC-SHA3. Yet end-to-end Phase-2 latency is nearly identical (1.5085 against 1.5030 ms), because the components common to both profiles dominate: the X25519 exchange contributes about 96 μs and PUF evaluation with fuzzy-extractor reproduction a further 1.4 ms.

The conclusion is that the lightweight profile is lighter in *gate count*, not in software execution time. A software comparison of the two hashes therefore does not support a claim about protocol-level performance, and the hardware-efficiency argument for SPONGENT must rest on area rather than on measured software latency.

7.5 Comparison with public-key baselines

Table 10 reports RSA, ECDSA, ECDH, and X25519 measured with the same library, on the same host, using the same clock as the protocol. Comparisons drawn from figures published for other hardware conflate protocol efficiency with processor class, and are avoided here.

Table 10: Same-platform public-key cost (OpenSSL 3.5, x86-64)

Scheme	Operation	Median (μs)	Mean (μs)
RSA-2048	sign	1048.6	1048.9
RSA-2048	verify	32.0	33.3
ECDSA P-256	sign	43.6	44.0
ECDSA P-256	verify	109.0	110.9
ECDH P-256	derive	91.0	92.6
X25519	keygen	65.6	66.3
X25519	derive	125.7	127.9

Two observations follow, both of which bear on how the literature reports these costs.

First, RSA-2048 *verification* costs approximately 33 μs . Figures in the millisecond range that are sometimes quoted for this operation are inconsistent with a small public exponent by two to three orders of magnitude, and any speedup multiplier derived from such a figure is unsound.

Second, this protocol is *comparable* to elliptic-curve schemes rather than faster by an order of magnitude. Each handshake performs one X25519 key generation and one derivation, approximately 190 μs of public-key work, against 91–155 μs for an ECDH or ECDSA exchange. This is the direct cost of making the ephemeral exchange mandatory, which forward secrecy requires; a protocol without it would be cheaper and would not offer the property.

The advantages of the present design are therefore properties rather than raw speed: no long-term secret is stored on the device, so physical capture yields no key material; no certificate infrastructure is required; peer authentication needs no ground-station round trip; the consequence of a capture

is bounded to the captured node’s own links; and the symmetric core is small enough ($1.0 \mu\text{s}$ hash, $2.1 \mu\text{s}$ MAC) that the hardware-area argument is unaffected.

7.6 Comparison with existing approaches

Tables 11–14 position the protocol against published PUF-based UAV schemes and against conventional public-key and pre-shared-key baselines.

Two caveats govern how these tables should be read. First, figures taken from the literature were obtained on the authors’ own platforms — typically ARM Cortex-A-class processors or Raspberry Pi boards — whereas ours are measured on an x86-64 host. Cross-platform latency comparisons therefore conflate protocol efficiency with processor class, and the platform column is given so that no such comparison is made implicitly. The public-key rows in Table 11 are the exception: those we re-measured ourselves on our own platform (§7.5), and only those are directly comparable with our own rows. Second, communication and storage costs are platform-independent and can be compared directly.

Table 11: Authentication latency. Only the rows marked *same platform* were measured on our host and are directly comparable with each other

Scheme	Platform	Device $\leftrightarrow$ server	Peer
RSA-2048 (sign+verify)	same platform	1.08 ms	—
ECDSA (sign+verify)	P-256 same platform	0.15 ms	—
ECDH P-256 (derive)	same platform	0.09 ms	—
Gope [2]	ARM (reported)	10–50 ms	—
Li [9]	ARM (reported)	45–80 ms	partial
Khan [12]	ARM (reported)	5–20 ms	—
Bera [3]	ARM (reported)	< 1 ms	< 1 ms
This work (sha3)	same platform	1.50 ms	0.42 ms
This work (spongent)	same platform	1.51 ms	0.42 ms

Our end-to-end figure includes one PUF evaluation and one fuzzy-extractor reproduction (≈ 1.4 ms of the 1.50 ms total, Table 8), which the public-key baselines do not perform at all. Excluding that device-specific work, the cryptographic handshake costs approximately 0.2 ms and is thus of the same order as an elliptic-curve exchange. We do not claim an order-of-magnitude latency advantage; §7.5 sets out why such claims are unsound on this evidence.

Table 12: Communication overhead for full mutual authentication (platform independent)

Scheme	Msgs	Device $\leftrightarrow$ server	Peer pair
RSA-2048 PKI (with cert)	2–4	768–1024 B	768 B
ECC-256 (with cert)	2–4	384–512 B	384 B
PSK (AES-GCM)	2	≈ 256 B	≈ 256 B
Gope [2]	3	≈ 200 B	—
Li [9]	4	≈ 300 B	—
This work, $N=5$	4	507 B	197 B
This work, $N=10$	4	777 B	197 B
This work, $N=20$	4	1317 B	197 B

Peer authentication at 197 B per pair is below every listed alternative, and it requires no ground-station involvement. The device–server figure grows with swarm size because M_4 carries one cre-

dential per peer: at $N = 10$ the credential package is 545 B of the 777 B total. Issuing credentials on demand rather than in bulk removes that growth at the cost of an extra round trip per new peer, and is the appropriate configuration for large swarms.

Table 13: Per-device storage for an N -UAV swarm. *Secret* counts material whose disclosure compromises authentication

Scheme	Stored on device	Total	Secret
RSA-2048 PKI	private key + cert chain	≈ 1.5 KB	256 B
ECC-256	private key + cert	≈ 0.4 KB	32 B
PSK	$N - 1$ pairwise keys	$32(N - 1)$ B	all
Gope [2]	helper data + identity	≈ 0.1 KB	none
This work	identity, challenge, helper data (4×32 B sketch + seed) + $N - 1$ peer credentials	$\approx 0.2 + 0.03N$ KB	$32(N - 1)$ B

The device holds no long-term secret of its own: identity, challenge and helper data are public by construction, and the master key exists only transiently, being regenerated from the PUF on demand. Per-pair credentials are secret, but each is confined to one link, so the disclosure of a device’s storage costs $N - 1$ links rather than the whole mesh — the property measured in §7.8. A pre-shared-key deployment stores the same quantity of pairwise material but derives it from no physical root, so a captured device also yields a usable long-term identity.

Table 14: Security properties. *Ephemeral* indicates forward secrecy from a per-session key exchange

Scheme	No stored secret	Mutual auth	Ephemeral (fwd sec)	GS-free peer	Capture blast radius
RSA-2048 PKI	No	Yes	Optional	Yes	1 device
ECC-256	No	Yes	Optional	Yes	1 device
PSK (AES-GCM)	No	Yes	No	Yes	whole swarm
Gope [2]	Yes	Yes	No	No	1 device
Li [9]	Yes	Yes	No	Partial	1 device
Khan [12]	Yes	Yes	Yes	No	1 device
Bera [3]	No	Yes	No	Yes	whole swarm
This work	Yes	Yes	Mandatory	Yes	$N - 1$ links

The distinguishing combination is GS-free peer authentication rooted in per-pair credentials, together with a mandatory rather than optional ephemeral exchange. Schemes that root peer authentication in a swarm-wide secret obtain low peer cost but lose all isolation on capture; schemes that route peer authentication through the ground station retain isolation but cannot operate while that link is unavailable. Making the ephemeral exchange mandatory rather than optional costs approximately $190 \mu\text{s}$ per handshake (Table 10) and is the reason we do not claim a latency advantage over elliptic-curve schemes.

7.7 Reliability of the fuzzy extractor

Table 15 reports the measured key-reproduction failure rate across bit-error rates and profiles, beside the analytic prediction $1 - (1 - \Pr[\text{Bin}(255, p) > t])^L$.

Table 15: Measured key-reproduction failure rate versus binomial prediction (30 devices per point)

BER	bch255-131-18		bch255-91-25		rep3-bch255-131-18	
	meas.	pred.	meas.	pred.	meas.	pred.
1%	0.000	9.3×10^{-11}	0.000	$< 10^{-12}$	0.000	$< 10^{-12}$
2%	0.000	5.1×10^{-6}	0.000	9.8×10^{-11}	0.000	$< 10^{-12}$
3%	0.033	1.2×10^{-3}	0.000	4.0×10^{-7}	0.000	$< 10^{-12}$
5%	0.167	2.1×10^{-1}	0.000	2.6×10^{-3}	0.000	$< 10^{-12}$
7%	0.800	8.9×10^{-1}	0.133	1.7×10^{-1}	0.000	2.9×10^{-8}
10%	1.000	1.0	1.000	9.7×10^{-1}	0.000	6.3×10^{-4}

Twenty of the twenty-one measured points lie within the 95% interval of the prediction. This agreement is the substantive evidence that error correction is actually performed: a decoder that returned its input unchanged would produce a curve independent of the code parameters, and could not track a binomial tail across four orders of magnitude.

The default profile reaches a failure rate near 1.2×10^{-3} at 3% bit-error rate. A failure probability below 10^{-15} is *not* attainable with a single-layer code at these parameters; the concatenated profile does reach it, at three times the PUF cost (3825 against 1020 response bits). This trade-off is reported as measured rather than asserted analytically. The default profile was chosen partly because its failure rate is observable: a rate of 10^{-15} cannot be measured in any feasible Monte-Carlo experiment, so a sweep against it would carry no information.

7.8 Security evaluation

The theorems of Section 5 are complemented by executing attacks against the running protocol. Each configuration declares the outcome it expects, and the run records whether that expectation held.

A negative control is essential to this argument. Reporting that every attack failed is unfalsifiable on its own, because it is equally consistent with a defective adversary. We therefore include an *ablation*: a variant in which one mechanism — the verification of the ground station’s authenticator on M_2 (design principle P3) — is disabled, with everything else unchanged, and we run the *same* attacker implementation against it. The ablation is a measurement instrument, not a deployment option; it exists to establish that the adversary detects a success when one is available.

Table 16: Attack outcomes. *Oracle* counts UAV responses to forged ground-station messages. The final row is the P3 ablation, in which the verify-before-respond rule is disabled to validate the adversary

Attack	Configuration	Att.	Acc.	Oracle	Expected	Matches
GS impersonation	full protocol	8	0	0	fail	yes
M_1 replay	full protocol	1	0	0	fail	yes
Credential sniffing	full protocol	10	0	0	fail	yes
GS impersonation	<i>ablation: no P3</i>	8	1	1	<i>succeed</i>	yes

Every outcome matches its declared expectation. Under the full protocol, eight forged ground-station messages produced no response of any kind: the UAV verifies the authenticator before performing any PUF-dependent computation, so no challenge–response pair is exposed and the modelling attack of Corollary 3 has no input to work from. Removing that single check in the ablation causes the same attacker code to elicit a response and obtain a challenge–response pair, which establishes two things: the adversary detects a success when one exists, so the zeros above are evidence rather than an artefact; and the verify-before-respond rule — not some incidental property of the implementation — is what closes the oracle.

Verbatim replay of M_1 inside the freshness window is refused by the nonce cache; the timestamp window alone would admit it. Passive inspection of M_4 across ten deliveries recovered no per-pair credential, the package being protected by authenticated encryption.

At the protocol level, capture of one UAV exposes only credentials incident to that node. In a four-UAV configuration this is three of six pairs, against all six under a swarm-wide credential.

8 Discussion

8.1 Scalability

Phase 2 is $O(N)$ in the swarm size and fully parallel across UAVs, since each UAV's handshake with the GS is independent. Phase 3 is $O(N^2)$: full-mesh peer authentication runs $N(N-1)/2$ exchanges (for example 45 for $N = 10$ and 190 for $N = 20$), but these too are mutually independent and can proceed in parallel, and a deployment that only needs neighbours may authenticate a subset of pairs. Because every operation is symmetric—a hash, a MAC, an AEAD, a KDF, and one ephemeral exchange—per-message cost is small and does not grow with the swarm.

8.2 Design Choices

Three choices give the protocol its character. First, deriving a *stable* key from the PUF means the drone stores no secret and can re-authenticate any number of times from a single enrollment, rather than consuming a limited pool of challenge–response pairs. Second, rooting Phase 3 in *per-pair* credentials—rather than one swarm-wide secret—means a captured node endangers only its own links, matching the capture threat that motivates PUFs in the first place. Third, making the ephemeral exchange *mandatory* rather than optional means forward secrecy is a property of every session, not a feature that can be silently skipped.

8.3 Noise Tolerance

The BCH(255,131,18) fuzzy extractor corrects up to 18 bit-flips against ≈ 7.65 expected flips in a 255-bit block at a 3% error rate. That margin is real but finite, and §7.7 measures where it ends: the default profile fails to reproduce mk_i for about one device in a thousand at 3%, roughly one in six at 5%, and almost always at 10%.

Three consequences deserve statement. First, a failure probability below 10^{-15} is not attainable with a single-layer code at these parameters; the concatenated profile reaches it at three times the PUF cost, and that is the trade-off rather than a property of the default configuration. Second, because the block length must exceed the $n - k = 124$ bits the sketch discloses, response blocks are 255 bits; a 128-bit block would leak almost all of itself and contribute essentially nothing to the key budget of Eq. (4). Third, a device that cannot reproduce its key does not authenticate incorrectly — it does not authenticate at all, and never transmits a first message. Availability, not security, is what degrades with environmental noise, and a deployment must budget for re-enrollment or a stronger profile accordingly.

8.4 Limitations and Future Work

The analysis here is at the protocol level; a mechanised end-to-end proof and a hardware prototype are natural next steps, as is an empirical study of the PUF's reliability across environmental extremes. Extending the model to dynamic swarm membership (join and leave during flight) and to mobility-induced link changes is left for future work.

9 Conclusion

We presented a four-phase PUF-based authentication protocol for UAV swarms. Its root of trust is a stable key distilled from each drone's PUF by a fuzzy extractor, so that no secret is ever stored on the device. Every handshake message is protected by a keyed MAC, and the raw PUF response never leaves the UAV, which closes the challenge-harvesting avenue open to challenge–response designs that transmit the response or a function of it. Peer authentication runs directly between UAVs, rooted in per-pair credentials delivered under authenticated encryption during the UAV–GS

handshake, so a captured node compromises only its own links. Every session key folds in a mandatory, authenticated ephemeral Diffie–Hellman secret, giving perfect forward secrecy. We proved six security theorems—mutual authentication in both directions, peer authentication, replay and man-in-the-middle resistance, physical-capture resistance, session-key secrecy, and perfect forward secrecy—by reduction to standard assumptions, and mirrored them in a Tamarin symbolic model. Built from a 160-bit hash, a keyed MAC, a lightweight AEAD, a KDF, and a single elliptic curve, the protocol is well matched to the resource constraints of aerial platforms.

A reference implementation on OMNeT++ 6.3 (§6) supports these claims with measurement. All 200 UAVs and all 3,800 peer pairs of a twenty-drone swarm authenticate, at 1.50 ± 0.003 ms per UAV–GS handshake and 0.418 ± 0.001 ms per peer pair over ten independent repetitions. Every primitive is validated against published reference values, including all 1,089 official Ascon-128a test vectors, and the fuzzy extractor’s measured failure rate tracks its binomial prediction across four orders of magnitude — the evidence that error correction genuinely occurs. Executed attacks confirm the analysis, and a control variant with the ground-station check removed demonstrates that the adversary detects success when it is present, so the negative results are falsifiable.

Two findings temper the performance narrative and are reported as measured. Same-platform baselines (§7.5) place RSA-2048 verification at $33 \mu\text{s}$, so millisecond-scale figures for that operation, and speedup multipliers derived from them, are unsound. On the same host this protocol is comparable to elliptic-curve schemes rather than faster by an order of magnitude, which is the direct cost of a mandatory ephemeral exchange. Its advantages are consequently properties rather than raw speed: no long-term secret on the device, no certificate infrastructure, peer authentication without a ground-station round trip, capture damage bounded to a node’s own links, and a symmetric core small enough to leave the hardware-area argument intact.

Remaining work is a mechanised proof discharged in Tamarin, an evaluation over a contention-aware MAC layer, and a hardware prototype.

Acknowledgment

The author thanks the Indian Institute of Information Technology Guwahati for providing the research infrastructure and Dr. Rakesh Matam for his guidance.

References

- [1] A. Mukherjee, S. Bose, A. Mukherjee, and S. Saha, “Securing UAV Flying Ad Hoc Wireless Networks: Authentication Development for Robust Communications,” *IEEE Access*, vol. 12, pp. 1–15, 2024.
- [2] P. Gope and B. Sikdar, “A PUF-based lightweight authentication and key agreement protocol for smart UAV networks,” *IET Communications*, vol. 16, no. 18, pp. 2136–2148, 2022.
- [3] T. Bera, S. Das, and A. K. Das, “Secure and Efficient Peer2Peer Authentication-Attestation Protocol for UAV Networks,” in *Proc. IEEE INFOCOM Workshop*, 2023, pp. 1–6.
- [4] National Technological University, “PUF-Based Lightweight Security Framework for Drone Networks,” DR-NTU Technical Report Series, 2023.
- [5] J. Delvaux, D. Gu, I. Verbauwhede, M. Hiller, and M.-D. Yu, “Cryptographic Key Generation from PUF Data Using Efficient Fuzzy Extractors,” *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 2025–2038, 2020.
- [6] L. Zhang, Y. Li, Z. Wang, and C. Chen, “SRAM-PUF Key Extraction Method and Performance Analysis Based on RS and BCH Codes,” *Electronics*, vol. 12, no. 4, p. 1012, 2023.
- [7] R. Maes, “Helper Data Masking for Physically Unclonable Function-Based Key Generation,” *Cryptology ePrint Archive*, Report 2019/688, 2019.

- [8] U. Chatterjee, R. S. Chakraborty, and D. Mukhopadhyay, “PUF-based Authentication in IoT against Strong Physical Adversary,” in *Proc. IEEE DATE*, 2022, pp. 1–6.
- [9] B. Li, H. Zhu, J. Shen, and T. Li, “PUF-Based Secure and Efficient Anonymous Authentication Protocol for UAV Towards Cross-Domain Environments,” *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12101–12115, 2023.
- [10] A. Alkatheiri, Y. G. Alghamdi, and S. U. Jan, “Review of Physically Unclonable Functions (PUFs): Structures, Models, and Algorithms,” *Frontiers in Electronics*, vol. 4, p. 1123456, 2023.
- [11] R. Aman, K. Z. Ghafoor, and S. K. Sood, “PUF based Lightweight Authentication and Key Exchange Protocol for IoT,” in *Proc. IEEE GLOBECOM*, 2022, pp. 1–6.
- [12] S. Khan, A. Irshad, S. A. Chaudhry, and M. Alazab, “A Provably Secure and Lightweight PUF-Based Authentication Scheme for 6G-Enabled UAV Networks,” *IEEE Internet of Things J.*, vol. 11, no. 5, pp. 8532–8543, 2024.
- [13] L. Xu, X. Wang, Q. Li, and H. Wang, “An Enhanced Event-Based Dynamic Authentication Protocol Leveraging Arbiter PUF for IoT Devices,” *IEEE Internet of Things J.*, vol. 10, no. 18, pp. 16123–16135, 2023.
- [14] T. Eisenbarth *et al.*, “Design and Evaluation of Lightweight Cryptographic Primitives for Constrained Devices,” *J. Cryptographic Engineering*, vol. 12, no. 3, pp. 245–260, 2022.
- [15] A. Bogdanov, M. Knežević, G. Leander, D. Toz, K. Varici, and I. Verbauwhede, “SPONGENT: A Lightweight Hash Function,” in *Proc. CHES 2011*, Springer LNCS, vol. 6917, pp. 312–325, 2011.
- [16] National Institute of Standards and Technology (NIST), “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS 202, 2015.
- [17] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “The Keccak Reference,” Version 3.0, 2011.
- [18] D. Dolev and A. Yao, “On the Security of Public Key Protocols,” *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [19] Y. Dodis, R. Ostrovsky, L. Reyzin, and A. Smith, “Fuzzy Extractors: How to Generate Strong Keys from Biometrics and Other Noisy Data,” *SIAM J. Comput.*, vol. 38, no. 1, pp. 97–139, 2008.
- [20] H. Krawczyk, “Cryptographic Extraction and Key Derivation: The HKDF Scheme,” in *Advances in Cryptology — CRYPTO*, 2010, pp. 631–648.
- [21] D. J. Bernstein, “Curve25519: New Diffie–Hellman Speed Records,” in *Public Key Cryptography — PKC*, 2006, pp. 207–228.
- [22] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, “Ascon v1.2: Lightweight Authenticated Encryption and Hashing,” *J. Cryptology*, vol. 34, no. 3, 2021.
- [23] M. Bellare and P. Rogaway, “Entity Authentication and Key Distribution,” in *Advances in Cryptology — CRYPTO*, 1993, pp. 232–249.
- [24] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN Prover for the Symbolic Analysis of Security Protocols,” in *Computer Aided Verification — CAV*, 2013, pp. 696–701.